07-structuring-code-examples/09/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
RSpec.configure do |config|
  config.include_context 'API helpers'
end
```

Next, let's turn to the other common use case for shared example groups: sharing *examples* instead of *context*.

Sharing Examples

One of the most powerful ideas in software is defining a single interface with multiple implementations. For example, your web app might need to cache data in a key-value store.[5] There are many implementations of this idea, each with its own advantages over the others. Because they all implement the same basic functionality of kv_store.store(key, value) and kv_store.fetch(key), you can choose the implementation that best fits your needs.

You don't even have to pick just one implementation. You might use one key-value store for production and a different one for testing. In production, you're likely to want a *persistent* store that keeps data around between requests. For testing, you can save time and complexity by using an in-memory key-value store.

If you're using more than one key-value store, it's important to have some confidence that they have the same behavior. You can write specs to test this behavior, and organize them using shared examples.

Without shared examples, you might start with the following spec for an in-memory HashKVStore:

07-structuring-code-examples/10/shared_examples/spec/hash_kv_store_spec.rb

```
require 'hash_kv_store'

RSpec.describe HashKVStore do
  let(:kv_store) { HashKVStore.new }

  it 'allows you to fetch previously stored values' do
    kv_store.store(:language, 'Ruby')
    kv_store.store(:os, 'linux')

    expect(kv_store.fetch(:language)).to eq 'Ruby'
    expect(kv_store.fetch(:os)).to eq 'linux'
  end

  it 'raises a KeyError when you fetch an unknown key' do
    expect { kv_store.fetch(:foo) }.to raise_error(KeyError)
  end
end
```

5. https://en.wikipedia.org/wiki/Key-value_database